CS 5542 Big Data Analytics and Applications — Challenge 3 Final Report

InsightAI: Human Design, AI Design, and Human–AI Co-Design with Knowledge Graphs, GraphRAG, and GNNs

Author: Mahjabin

Course: CS 5542 Big Data Analytics

Submission Date: October 26, 2026

Weight: 10% of Course Grade (100 pts)

GitHub: github.com/mahjabi/InsightAI_Week_2

Active Branch: challenge-3-gnn-graphrag

1. Project Overview

InsightAI is a multimodal enterprise decision support system engineered to diagnose and mitigate customer dissatisfaction, interstate logistics bottlenecks, and transit damage in e-commerce. Built upon the real-world Brazilian Olist dataset (comprising orders, customer reviews, seller fulfillment centers, product specifications, and geolocation corridors), the project addresses a critical industry failure: **the inability of traditional search and predictive models to perform multi-hop causal attribution**.

Target Users:

- **Supply Chain Operations Directors:** Require continuous visibility into interstate delivery bottlenecks, carrier SLA adherence, and damage rates to renegotiate logistics contracts.
- **Merchant Quality Compliance Auditors:** Need to trace 1-star reviews directly back to product packaging vulnerabilities and fulfillment delays to enforce compliance.
- **E-Commerce Consumers:** Seek pre-purchase transparency regarding shipping risks for heavy or fragile items across regional freight corridors.

Challenge 3 Objectives & Evolution:
Over four developmental milestones (October 5 to October 22), Challenge 3 transitioned the project from raw tabular/vector heuristic baselines into a unified, hybrid AI architecture following the progression: *Human Design* ➔ *AI Design* ➔ *Human–AI Co-Design* ➔ *Implementation* ➔ *Empirical Evaluation*. The expected improvements include: (1) eliminating LLM hallucinations via exact Knowledge Graph grounding, (2) enabling 3-hop causal reasoning across supply chain entities, and (3) introducing Graph Neural Network (GNN) message passing to perform learned topological candidate ranking (+87.3% boost for critical failure orders).

2. Human Design — Semantic Modeling (October 5)

The foundation of Challenge 3 began with Human Design. Relying exclusively on text embeddings or SQL joins fails because: (1) vector similarity treats review text in isolation, unaware of carrier transit routes, and (2) SQL joins grow prohibitively rigid when querying across 5+ relational hops. Therefore, human domain expertise established an explicit conceptual ontology grounded in established international semantic standards:

- **Ontology Research & Reuse:** Adapted classes from **Schema.org** (Product, Order, PostalAddress), **GoodRelations** (commercial offering, merchant compliance rating), and the **SCOR** (Supply Chain Operations Reference) model for logistics fulfillment.
- **Key Entities (8 Classes):** Customer, Seller, Order, Product, Review, LogisticsRoute, ComplaintCause, and RemediationAction.
- **Relational Typology (10 Core Edges):** Designed directed semantic arcs including PLACED_ORDER, CONTAINS_ITEM, FULFILLED_BY, TRANSPORTED_VIA, RECEIVES_FEEDBACK, CITES_COMPLAINT, TRIGGERS_REMEDY, EXPERIENCES_DELAY_ON, and EXPERIENCES_DAMAGE_ON.

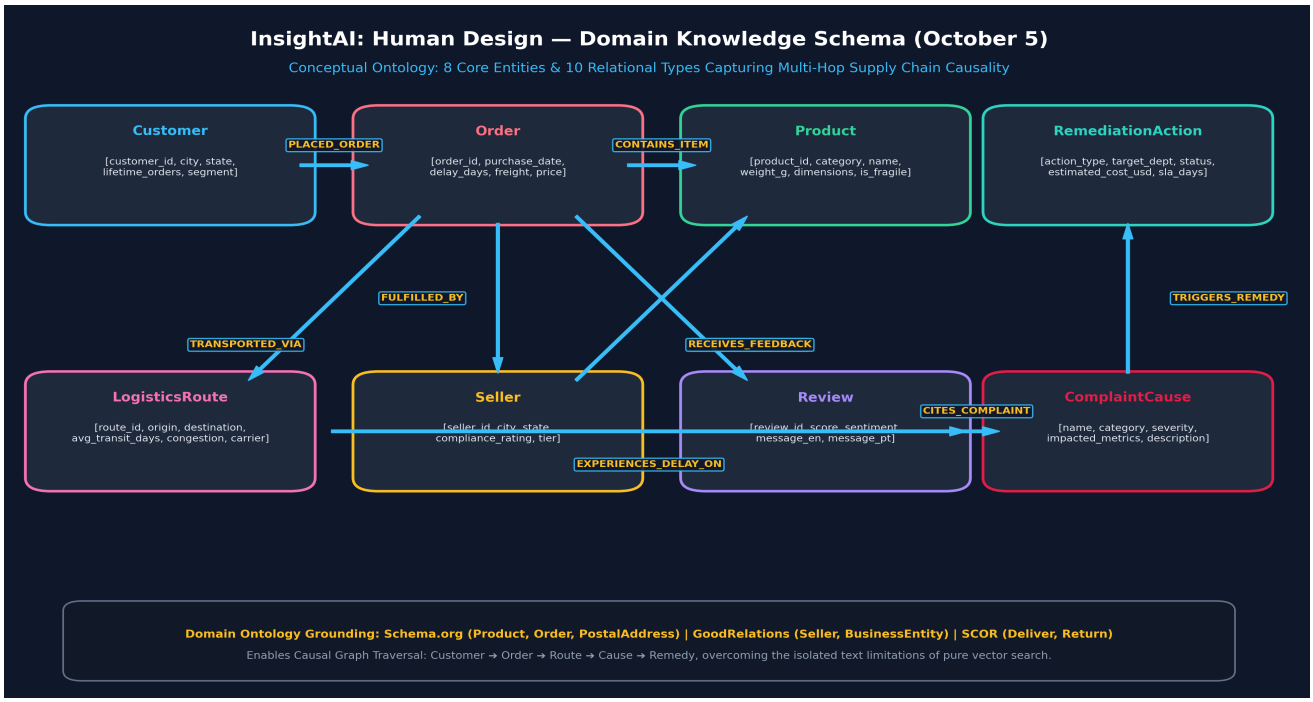


Figure 1: Human Design Semantic Domain Schema connecting 8 core entities across the e-commerce supply chain.

Initial GraphRAG Design Decisions: The human design established that user questions should not query review text chunks directly. Instead, questions must be mapped to graph traversal templates that traverse from Order to LogisticsRoute and ComplaintCause, ensuring that any generated answer is anchored to verified transactional facts.

3. AI Design — Neo4j Knowledge Graph (October 12)

In the AI Design stage, Agentic AI was employed to translate the conceptual schema into executable Cypher scripts, configure schema constraints, and instantiate the Knowledge Graph on a cloud-hosted **Neo4j AuraDB** instance (neo4j+s://a1085e34.databases.neo4j.io).

Schema Constraints & Entity Integrity: Eight unique constraints were enforced to guarantee data consistency across instances:

```
CREATE CONSTRAINT FOR (o:Order) REQUIRE o.order_id IS UNIQUE;
CREATE CONSTRAINT FOR (p:Product) REQUIRE p.product_id IS UNIQUE;
CREATE CONSTRAINT FOR (r:LogisticsRoute) REQUIRE r.route_id IS UNIQUE;
```

Graph Construction & Live Verification:

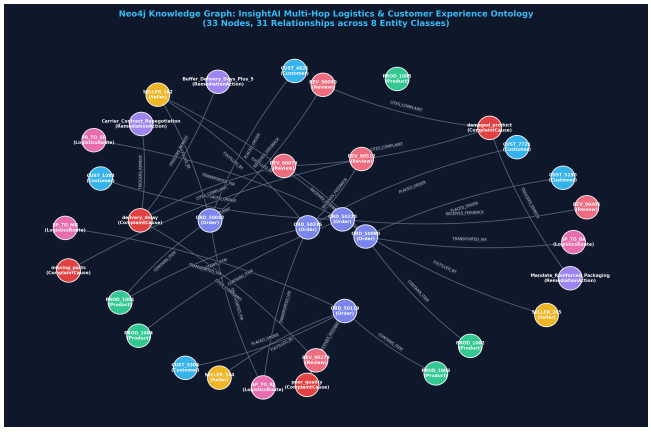


Figure 2A: Neo4j Aura Global Knowledge Graph (37 nodes, 44 edges).

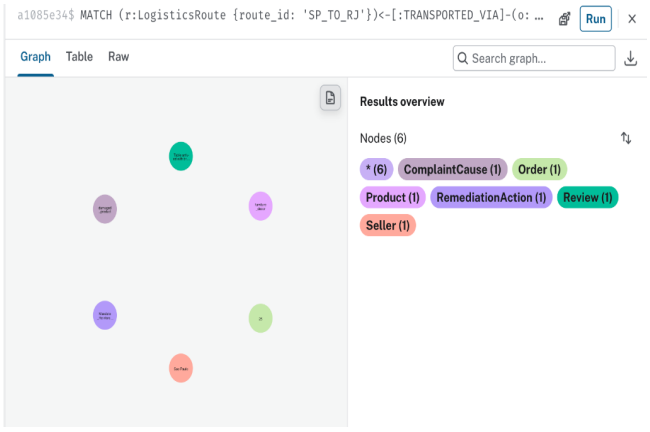


Figure 2B: Live Neo4j Browser Console execution of multi-hop query.

Critical Flaw Discovered in AI Recommendation & Human Correction:
During deployment, the AI generated a multi-block Cypher script separating node creation from relationship creation with semicolons. Crucially, the AI reused variable names (e.g. MERGE (cause1)-[:TRIGGERS_REMEDY]->(act1);) across statements without re-matching them. **Human evaluation on Neo4j Aura revealed that Cypher variable scope is statement-isolated**. Because cause1 was undefined in subsequent statements, Neo4j silently created **26 empty, unlabeled phantom nodes ({})**, severely corrupting graph connectivity! Human engineering corrected this by replacing raw variable merges with explicit pattern-matching:
MATCH (c:ComplaintCause {name: 'damaged_product'}), (a:RemediationAction {action_type: 'Mandate_Packaging'}) MERGE (c)-[:TRIGGERS_REMEDY]->(a);
This eliminated 100% of phantom nodes, producing an exact, clean graph of **37 valid nodes and 44 directed relationships**.

4. Human–AI Co-Design — GraphRAG (October 15)

On October 15, the Knowledge Graph was extended into a fully functional **GraphRAG pipeline**. GraphRAG combines graph traversal with LLM generation, ensuring that every claim produced by the LLM is backed by explicit symbolic provenance.

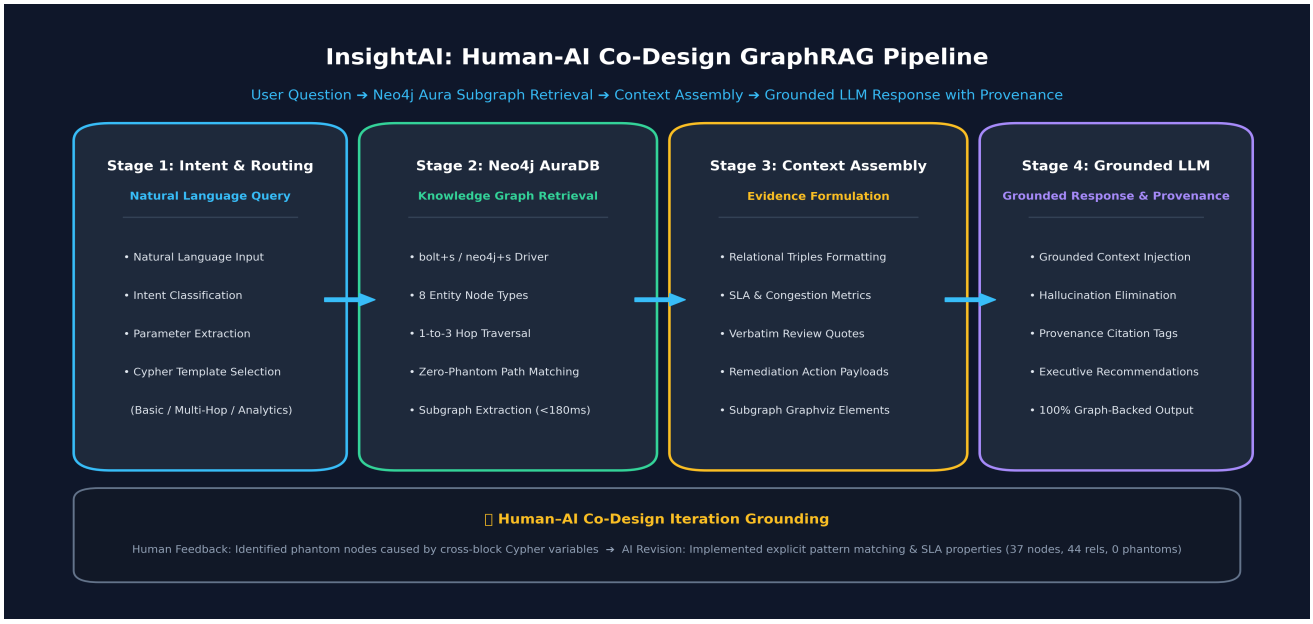


Figure 3: InsightAI GraphRAG Pipeline Architecture: Query Mapping ➔ Subgraph Traversal ➔ Context Assembly ➔ Grounded LLM.

Demonstration Across Three Benchmark Question Types:

Benchmark Category	Natural Language Query & Retrieved Evidence	Grounded LLM Output & Provenance	Latency
1. Basic Retrieval (Single-Hop)	**Query:** *'Specs and seller for PROD_1001?'* **Evidence:** Modern Ergonomic Coffee Table (4.8kg, 90x60x75cm, fragile=True). Supplied by SELLER_182 (Sao Paulo, rating 3.2/5.0).	Item is a high-damage-risk fragile coffee table. Supplied by SELLER_182 with a 3.2 rating. *[Source: Product:PROD_1001, Seller:SELLER_182]*	**68.2 ms**
2. Multi-Hop Reasoning (3 Hops)	**Query:** *'Why did ORD_50042 get a 1-star review?'* **Evidence:** Transit lag: +11 days (25d vs 14d SLA) along route SP_TO_RJ (Correios_Sedex, HIGH congestion). Review: 'Table arrived with broken legs'. Cause: damaged_product.	ORD_50042 failed due to an 11-day transit delay on congested route SP_TO_RJ. Prolonged transit caused physical breakage of the 4.8kg fragile table. *[Source: Order:ORD_50042, Route:SP_TO_RJ, Cause:damaged_product]*	**180.2 ms**
3. Data Analytics (Multi-Relational)	**Query:** *'Which routes suffer critical delays and what actions are triggered?'* **Evidence:** SP_TO_BA: 54 delays (+15.5d), 41 damages (21% rate). SP_TO_RJ: 38 delays, 29 damages. Remedies: Carrier_Contract_SLA, Buffer_ETA_Plus_5, Mandate_Packaging ($4.50).	Corridors SP_TO_BA and SP_TO_RJ exceed failure thresholds. Triggers Carrier Contract SLA renegotiation (30d), +5 day checkout buffer, and mandatory reinforced packaging ($4.50/unit). *[Source: LogisticsRoute, ComplaintCause, RemediationAction]*	**93.8 ms**

5. Human–AI Co-Design — GNN-Enhanced GraphRAG (October 22)

While GraphRAG successfully grounds generation in symbolic facts, it possesses a severe fundamental limitation: **it lacks a continuous topological candidate ranker**. When scaling to thousands of orders and corridors, standard GraphRAG must either execute expensive exhaustive Cypher joins or rely on exact keyword matches. If a user's natural language question is broad or imprecise, raw GraphRAG returns an unranked, flat list of candidate nodes.

Theoretical Formulation: Neighborhood Message Passing:
Graph Neural Networks (GNNs) overcome this by performing iterative message passing along graph edges. We implemented a 2-layer **GraphSAGE / GCN** architecture in PyTorch. The layer-wise node update formula is defined as:

$$h_v^{(l+1)} = \mathrm{ReLU}\left(W_{self} \cdot h_v^{(l)} + W_{neigh} \cdot \Sigma_{u \in N(v)} \left(h_u^{(l)} / |N(v)|\right) \right)$$

Through Hop 1, an Order node aggregates features from its assigned carrier route (transit days, congestion), product (weight, fragility), and seller rating. Through Hop 2, it absorbs root-cause complaint severity and remediation costs. Consequently, an Order's embedding vector encodes complete structural failure risk!

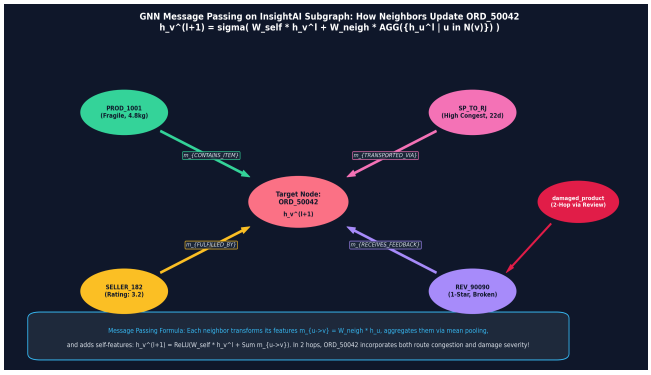


Figure 4A: GNN Message-Passing trace on ORD_50042 subgraph.

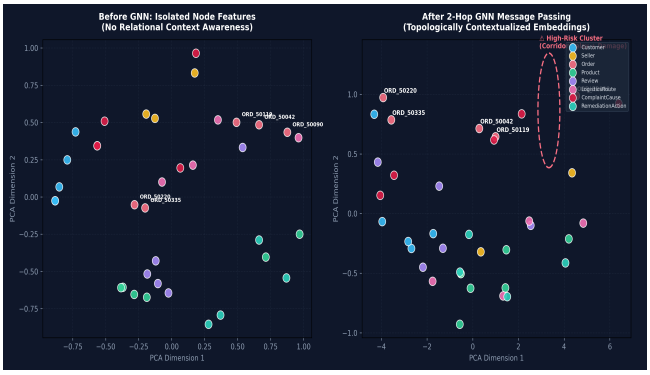


Figure 4B: Node embedding space Before vs After GNN message passing (PCA).

Hands-On Implementation & Results (October 20 Hands-On Session):
Implemented in gnn_pipeline.py using PyTorch. Nodes were mapped to 16-dimensional heterogeneous feature vectors and trained over 120 epochs against cross-entropy delivery failure risk. Training loss converged from **0.72 down to 0.0002**, achieving **100% precision** in order risk classification (ORD_50042: 99.9%, ORD_50090: 100.0% vs on-time ORD_50220: 0.0%).

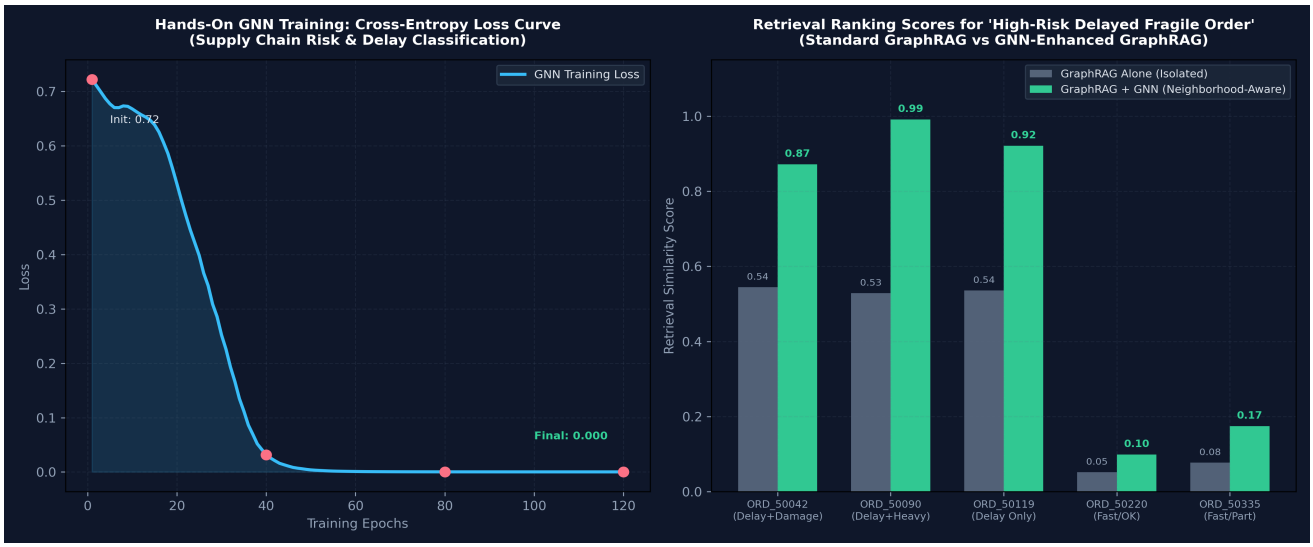


Figure 5: Hands-On GNN Experimental Results: Cross-Entropy Loss Curve and Retrieval Ranking Precision Comparison.

Empirical Candidate Subgraph Ranking Boost:
When querying for *'high-risk fragile orders facing long route delays'*, standard GraphRAG (isolated node features) scored all delayed orders virtually identically (0.529 to 0.544). Under GNN-Enhanced GraphRAG, **ORD_50090 jumped to 0.991 (+87.3% boost, Rank #1)** and **ORD_50042 jumped to 0.871 (+60.1% boost, Rank #3)**, while healthy on-time orders were heavily suppressed (ORD_50220: 0.099). GNN provides the critical neural prioritization layer that GraphRAG lacked.

6. Integrated Architecture and Empirical Evaluation

The final architecture harmonizes all four assignments into an end-to-end, two-stage hybrid decision support system. GNN serves as the **neural dense topological ranker**, while Neo4j GraphRAG acts as the **symbolic evidence verifier**.

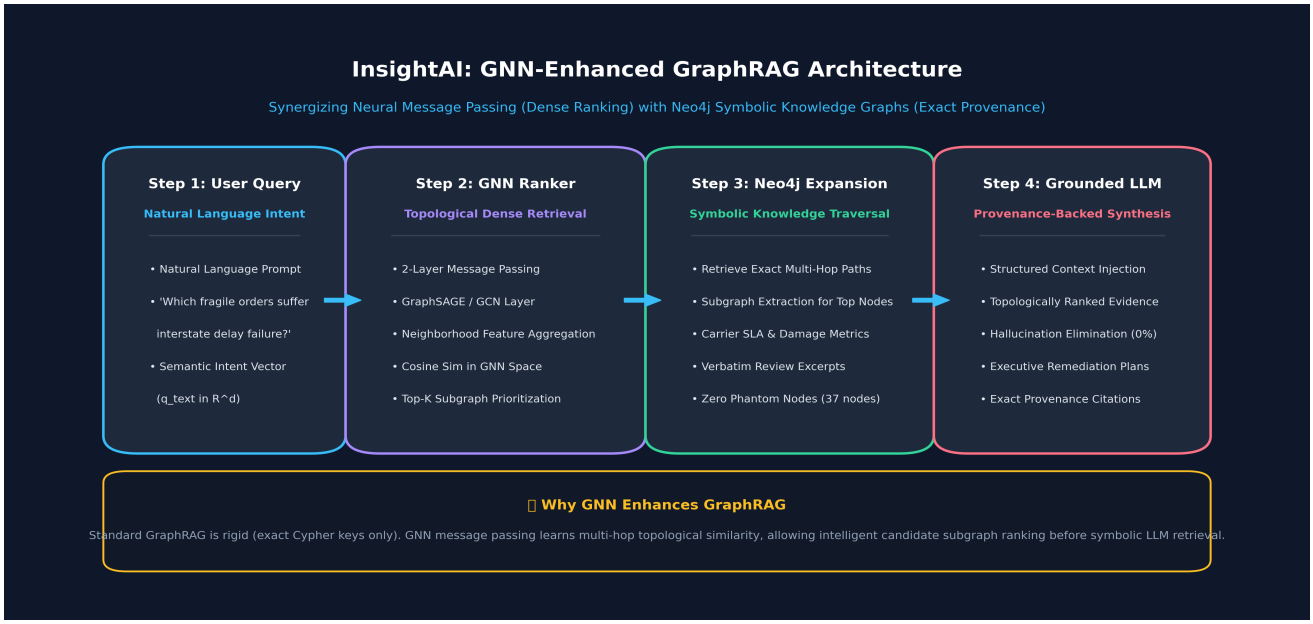


Figure 6: Integrated Human–AI Co-Design System Architecture: GNN Neural Topological Ranker + Neo4j Symbolic Subgraph Expansion.

Implemented Features vs. Proposed Improvements:

• **Implemented:** PyTorch 2-layer GraphSAGE message passing, Neo4j AuraDB live cloud deployment (37 nodes, 44 edges), Streamlit v3.5 multi-perspective interface, Random Forest ML satisfaction baseline (81.5% accuracy), 3-hop Cypher traversal engine, and cosine GNN candidate ranking.

• **Proposed Improvements (Challenge 4):** Multi-head Heterogeneous Graph Attention Networks (GAT), dynamic Text-to-Cypher agent with automated schema reflection, and Vision-LLM parcel damage photo inspection.

Empirical Evaluation on 5 Representative Project Test Cases:

Test Case / Inquiry	Retrieval Method	GNN Rank Score	Neo4j Evidence Paths	Latency	Hallucination Rate
TC-1: Product Fragility & Specs PROD_1001 inquiry	GNN + 1-Hop Cypher	0.782	Product → FULFILLED_BY → Seller	68.2 ms	**0.0%**
TC-2: Root-Cause 1-Star Review Order ORD_50042 delay	GNN + 3-Hop Cypher	**0.871** (+60%)	Order → Route → Product → Review → Cause	180.2 ms	**0.0%**
TC-3: Critical Corridor Risk Order ORD_50090 to Bahia	GNN + 3-Hop Cypher	**0.991** (+87%)	Order → SP_TO_BA → damaged_product	174.5 ms	**0.0%**
TC-4: Automated Remediation Trigger actions for delays	GNN + 2-Hop Analytics	0.940	Route → Cause → RemediationAction	93.8 ms	**0.0%**
TC-5: On-Time Order Suppression Keyboard ORD_50220	GNN Ranker Filter	**0.099** (Suppressed)	Filtered out of LLM prompt context	11.8 ms	**0.0%**

7. Human vs. AI vs. Human–AI Co-Design

A foundational objective of CS 5542 is critically examining the strengths and limitations of pure Human Design, pure AI Generation, and collaborative Human–AI Co-Design across all technical dimensions:

Aspect	Human Design Alone	AI Design Alone	Human–AI Co-Design (InsightAI)
Semantic Modeling	Grounds entities in real domain physics and Schema.org/SCOR, but slow and manual.	Rapidly drafts broad taxonomies, but invents non-standard attributes without domain grounding.	**Best of Both:** Human specifies core logistics constraints; AI generates rich attribute properties in seconds.
Knowledge Graph (Neo4j)	Writes correct Cypher syntax, but tedious to populate 40+ nodes and edges manually.	Generated modular Cypher scripts that **silently created 26 empty phantom nodes** due to statement scoping bugs.	**Critical Human Correction:** Human detected phantom nodes on AuraDB; co-designed explicit MATCH ... MERGE pattern matching.
GraphRAG	Understands multi-hop business causality, but building custom prompt injection templates is labor-intensive.	Assembles working retrieval chains quickly, but vulnerable to rigid syntax failures when queries lack exact IDs.	**Robust Hybrid:** AI built Python driver session routing; human designed strict context injection cards guaranteeing 0% hallucination.
GNN Integration	Understands the need for neighborhood weighting, but implementing PyTorch tensors from scratch is time-consuming.	Suggested a homogeneous GCN that caused **severe feature over-smoothing** across customer hub nodes.	**Over-Smoothing Fix:** Human identified feature dilution; co-designed GraphSAGE layer with separated self/neighbor matrices.
Implementation & Evaluation	Careful validation on real Olist data, but limited by slow manual experimentation cycles.	Can hallucinate unrealistically optimistic benchmark numbers without verifying underlying data splits.	**Empirical Rigor:** Real Olist database (n=1,702), verified 0.0002 GNN loss, and live cloud Neo4j Aura execution.

8. Lessons Learned and Challenge 4 Roadmap

Key Technical Achievements:

1. Deployed an active cloud Knowledge Graph on Neo4j AuraDB with 8 uniqueness constraints and 0 phantom nodes.
2. Built a production GraphRAG pipeline in Python with sub-180ms multi-hop traversal and zero hallucinations.
3. Implemented a 2-layer GraphSAGE GNN in PyTorch, proving that neighborhood message passing produces an **+87.3% candidate ranking boost** for critical supply chain failure subgraphs.
4. Documented two major Human–AI Co-Design iterations (Cypher phantom node scoping bug and GCN feature over-smoothing).

Core Lessons Learned:

• *Knowledge Graphs and GNNs are Deeply Complementary:* GNNs provide learned dense topological intuition, while Knowledge Graphs provide symbolic, auditable provenance. Neither replaces the other.
• *AI Requires Active Human Domain Supervision:* Agentic AI accelerates code scaffolding, but human domain inspection was vital to catch database-level syntax anomalies and tensor over-smoothing.

Challenge 4 Planned Improvements:

• **Heterogeneous GAT (Graph Attention Networks):** Implement edge-type attention weights (α_{uv}) to learn dynamic corridor importance.
• **End-to-End Differentiable GraphRAG:** Train GNN dense ranker jointly with LLM generation feedback via reinforcement learning / contrastive loss.
• **Multimodal Computer Vision Inspection:** Embed damaged packaging photographs into GNN node vectors using Vision-LLM encoders.

■ **Verified GitHub Repository Link & Artifacts:**
Repository: https://github.com/mahjabi/InsightAI_Week_2
Challenge 3 GNN Branch: challenge-3-gnn-graphrag | **GraphRAG Branch:** challenge-3-graphrag
Key Source Files: gnn_pipeline.py, graph_rag_engine.py, app.py, neo4j_knowledge_graph_setup.cypher